

Module Name: Calculator

Design Engineers: Suneet Tumma

Verification Engineers: Suneet Tumma

Test Bench File: calc_tb_top.sv

Functionality	Basic Computation
☐ Normal addition with no overflow	Write the numbers 123 and 3425 to address 0 and 1 on SRAM A. Write the numbers 856874 and 12709 to address 0 and 1 on SRAM B. Start calculator and verify result. The expected output is 856997 on SRAM A and 16134 on SRAM B.
☐ Normal Addition with overflow (MAX + MAX)	Write the number 0xFFFFFFFF to addresses 0 and 1 on both SRAMs and start calculator and verify result. The expected output is 0xFFFFFFF0 on both SRAMs.

Functionality	Edge Cases
☐ 0 + 0	Write 0 to address 0 and 1 on both SRAMs. Start the calculator and verify the output. The expected output is 0 on both SRAMs.
☐ 0 + MAX	Write 0 to address 0 and 1 on SRAM A and max value of 0xFFFFFFFF to address 0 and 1 on SRAM B. Start the calculator and verify output. The expected output is 0xFFFFFFFF on both SRAMs.

Functionality	Data Flow and SRAM Integrity
☐ SRAM Read/Write	Manually choose write data and address and assert write enable signal. Then choose the same address for read and verify that the written and read data are equal. Used the number: 3214532 for this test.

Functionality	Timing & Reset Tests (Assertion)
☐ Reset functionality	<pre>property RESET; @(posedge clk) calc_if.reset => (state === S_IDLE); endproperty</pre>
☐ Verify the valid current read and write addr	<pre>property CHECK_CURRENT_ADDRS; @(posedge clk) disable iff (\$isunknown(state) state != S_IDLE state != S_END) (calc_if.curr_rd_addr < calc_if.read_end_addr && calc_if.curr_rd_addr > calc_if.read_start_addr) && (calc_if.curr_wr_addr < calc_if.write_end_addr && calc_if.curr_wr_addr > calc_if.write_start_addr); endproperty</pre>
☐ Buffer Toggling	<pre>property BUFFER_TOGGLING; @(posedge clk) (state == S_READ) => \$changed(calc_if.loc_sel); endproperty</pre>

Functionality	Random Constrained Test (Coverage)
☐ give random input	Randomize SRAM and read/write start/end addresses. Start calculator and verify results. Repeat 50 times.

Functionality	FSM Coverage
☐ FSM coverage	Wait for state to reach S_READ and then assert reset signal. Repeat with S_ADD and S_WRITE to ensure all state transitions are covered.